

南開大學
Nankai University

计算机学院

计算机组成原理创新实验

8 位 AI 运算器设计 week1

姓名：赵宇哲

学号：2411094

专业：计算机科学与技术

2026 年 4 月 30 日

目录

1 实验目的 **2**

2 实验原理 **2**
2.1 CNN 基础流程 2
2.2 8 位定点运算说明 2

3 一点疑惑 **3**
3.1 关于 ReLU 模块的想法 3

4 基础模块设计 **3**
4.1 8 位无符号乘法器设计 3
4.2 8 位无符号加法器设计 3
4.3 ReLU 激活模块设计 4
4.4 除以 4 模块设计 4

5 Verilog 代码实现 **4**
5.1 8 位无符号乘法器代码 4
5.2 8 位无符号加法器代码 5
5.3 ReLU 激活模块代码 5
5.4 除以 4 模块代码 5

6 功能仿真与结果分析 **6**
6.1 乘法器仿真结果分析 6
6.2 加法器仿真结果分析 6
6.3 ReLU 模块仿真结果分析 6
6.4 除以 4 模块仿真结果分析 8

7 实验总结 **8**

A Verilog 源代码 **9**
A.1 8 位无符号乘法器代码 9
A.2 8 位无符号加法器代码 9
A.3 ReLU 激活模块代码 9
A.4 除以 4 模块代码 10
A.5 乘法器测试文件代码 10
A.6 加法器测试文件代码 11
A.7 ReLU 测试文件代码 11
A.8 除以 4 测试文件代码 12

1 实验目的

本实验面向边缘端 AI 计算场景，设计用于卷积神经网络基础计算流程的 8 位定点运算模块。第一周主要完成卷积、激活和池化过程中所需的基础运算单元设计，包括 8 位无符号乘法器、8 位无符号加法器、ReLU 激活模块以及除以 4 模块，并通过 Vivado 行为仿真验证各模块功能的正确性。

通过本周实验，需要掌握以下内容：

1. 理解 CNN 中卷积、激活和池化的基本计算过程；
2. 掌握 8 位无符号定点数在硬件运算中的表示方式；
3. 使用 Verilog HDL 完成基础运算模块设计；
4. 使用 Vivado 对各模块进行功能仿真；
5. 分析仿真波形，验证模块输出结果是否符合理论计算。

2 实验原理

2.1 CNN 基础流程

卷积神经网络可以理解为一种逐层提取图像特征的计算结构。在图像识别任务中，输入图像通常先经过卷积层提取局部特征，再经过激活层增强非线性表达能力，最后通过池化层压缩数据规模，减少后续计算量。

本实验模拟智能宠物喂食器中猫狗面部特征提取的核心计算流程。整体处理流程为：

$$5 \times 5\text{输入矩阵} \rightarrow 3 \times 3\text{卷积} \rightarrow \text{ReLU 激活} \rightarrow 2 \times 2\text{平均池化}$$

第一周暂不完成完整链路设计，而是先完成其中所需的基础硬件模块，包括乘法器、加法器、ReLU 模块和除以 4 模块。

2.2 8 位定点运算说明

在边缘端 AI 设备中，芯片面积和功耗通常受到限制。相比 32 位浮点运算，8 位定点运算具有资源占用少、计算速度快、功耗低等优点，适合部署在轻量级 AI 硬件中。

本实验中输入数据采用 8 位无符号整数表示，取值范围为：

$$0 \sim 255$$

由于卷积和池化过程中存在乘法与累加操作，部分模块输出位宽需要扩展，以避免计算结果溢出。例如，8 位无符号数相乘的最大值为：

$$255 \times 255 = 65025$$

因此乘法器输出需要设置为 16 位。8 位无符号数相加的最大值为：

$$255 + 255 = 510$$

因此加法器输出需要设置为 9 位。

3 一点疑惑

3.1 关于 ReLU 模块的想法

在一般卷积神经网络中，ReLU 激活函数的表达式为：

$$ReLU(x) = \max(0, x)$$

其作用是当输入为负数时输出 0，当输入为正数时保留原值。因此，通用 ReLU 模块通常需要判断输入数据是否小于 0。

但是，本实验第一周设计的输入数据采用 8 位无符号整数，取值范围为：

$$0 \leq x \leq 255$$

因此输入数据不可能出现负数。在该条件下，ReLU 函数中的“负数置零”分支不会被触发，ReLU 模块可以简化为：

$$y = x$$

所以本实验中的 ReLU 模块采用直通结构，即直接将输入信号赋值给输出信号。该设计不是忽略 ReLU 功能，而是基于无符号输入范围作出的硬件优化，可以减少比较器和选择器的使用，降低逻辑资源占用。

需要注意的是，该简化只适用于当前无符号输入条件。若后续卷积核权重或卷积结果扩展为有符号数，则卷积输出可能出现负值，此时 ReLU 模块需要改为带符号判断结构，即当输入小于 0 时输出 0，否则输出原值。

4 基础模块设计

4.1 8 位无符号乘法器设计

8 位无符号乘法器用于卷积层中像素值与卷积核权重的乘法运算。模块输入为两个 8 位无符号数，输出为 16 位无符号数。

模块接口设计如下：

- 输入：`a[7:0]`，`b[7:0]`
- 输出：`y[15:0]`

由于 8 位乘 8 位的最大结果为 65025，需要 16 位输出才能完整表示结果。

4.2 8 位无符号加法器设计

8 位无符号加法器用于卷积累加以及池化求和过程。模块输入为两个 8 位无符号数，输出为 9 位无符号数。

模块接口设计如下：

- 输入：`a[7:0]`，`b[7:0]`

- 输出：`y[8:0]`

输出设置为 9 位，可以避免两个 8 位最大值相加时发生溢出。

4.3 ReLU 激活模块设计

根据第 3.1 节的分析，本实验第一周的输入数据采用 8 位无符号整数，输入范围为：

$$0 \leq x \leq 255$$

因此输入信号不存在负数。在该条件下，通用 ReLU 函数中的负数置零分支不会被触发，ReLU 模块可以简化为直通结构。

本模块输入为 8 位无符号信号 `x[7:0]`，输出为 8 位无符号信号 `y[7:0]`，其逻辑关系为：

$$y = x$$

模块接口设计如下：

- 输入：`x[7:0]`
- 输出：`y[7:0]`

该模块不需要比较器和多路选择器，只需将输入信号直接连接到输出信号。这样既满足当前无符号输入条件下的 ReLU 功能，又减少了硬件逻辑资源占用，符合边缘端 AI 运算器低资源、低功耗的设计目标。

4.4 除以 4 模块设计

除以 4 模块用于实现 2×2 平均池化。2×2 平均池化需要先对 4 个输入数据求和，再除以 4 得到平均值。

4 个 8 位数据相加的最大值为：

$$255 + 255 + 255 + 255 = 1020$$

因此求和输入至少需要 10 位。由于除以 4 等价于二进制右移 2 位，所以本实验采用右移操作实现除法：

$$avg = sum >> 2$$

这种实现方式比直接使用除法运算更节省硬件资源。

5 Verilog 代码实现

5.1 8 位无符号乘法器代码

```
1 module mul8u (
2     input  [7:0] a,
```

```
    input  [7:0] b,
    output [15:0] y
);

assign y = a * b;

endmodule
```

5.2 8 位无符号加法器代码

```
module add8u (
    input  [7:0] a,
    input  [7:0] b,
    output [8:0] y
);

assign y = a + b;

endmodule
```

5.3 ReLU 激活模块代码

```
module relu_u8 (
    input  [7:0] x,
    output [7:0] y
);

assign y = x;

endmodule
```

5.4 除以 4 模块代码

```
module div4_u10 (
    input  [9:0] x,
    output [7:0] y
);

assign y = x >> 2;

endmodule
```

6 功能仿真与结果分析

6.1 乘法器仿真结果分析

图6.1为 8 位无符号乘法器的仿真波形。仿真中选取了多组输入数据进行测试，包括普通数值和最大边界值。

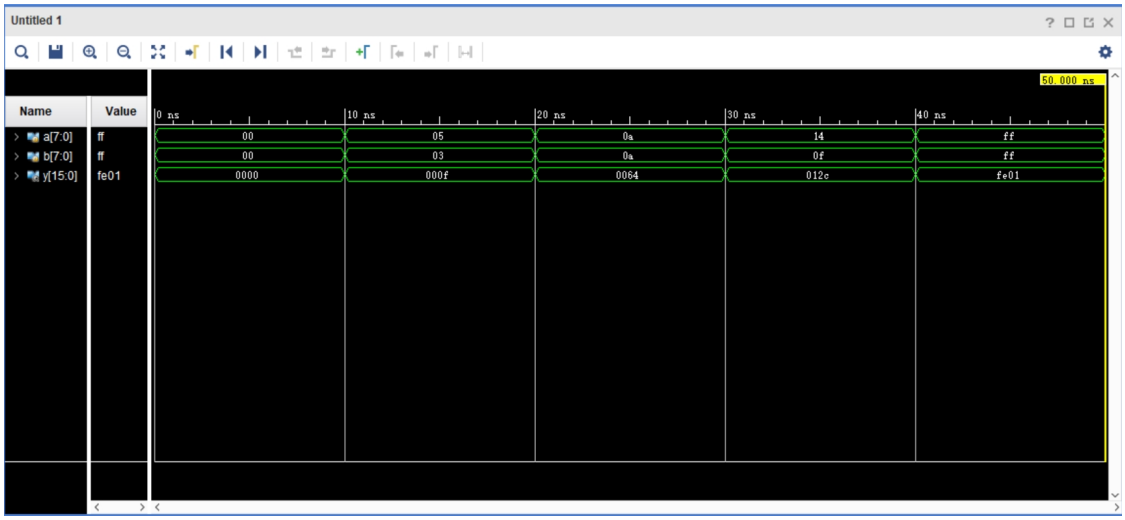


图 6.1: 8 位无符号乘法器仿真波形

由仿真结果可知，当输入为 `a=05`、`b=03` 时，输出为 `000f`，对应十进制结果为 15，符合：

$$5 \times 3 = 15$$

当输入为 `a=ff`、`b=ff` 时，输出为 `fe01`，对应十进制结果为 65025，符合：

$$255 \times 255 = 65025$$

因此，8 位无符号乘法器功能正确。

6.2 加法器仿真结果分析

图6.2为 8 位无符号加法器的仿真波形。
由仿真结果可知，当输入为 `a=05`、`b=03` 时，输出为 `008`，对应十进制结果为 8，符合：

$$5 + 3 = 8$$

当输入为 `a=ff`、`b=ff` 时，输出为 `1fe`，对应十进制结果为 510，符合：

$$255 + 255 = 510$$

由于输出位宽设置为 9 位，因此能够正确表示加法溢出后的结果。仿真结果说明加法器功能正确。

6.3 ReLU 模块仿真结果分析

图6.3为 ReLU 激活模块的仿真波形。

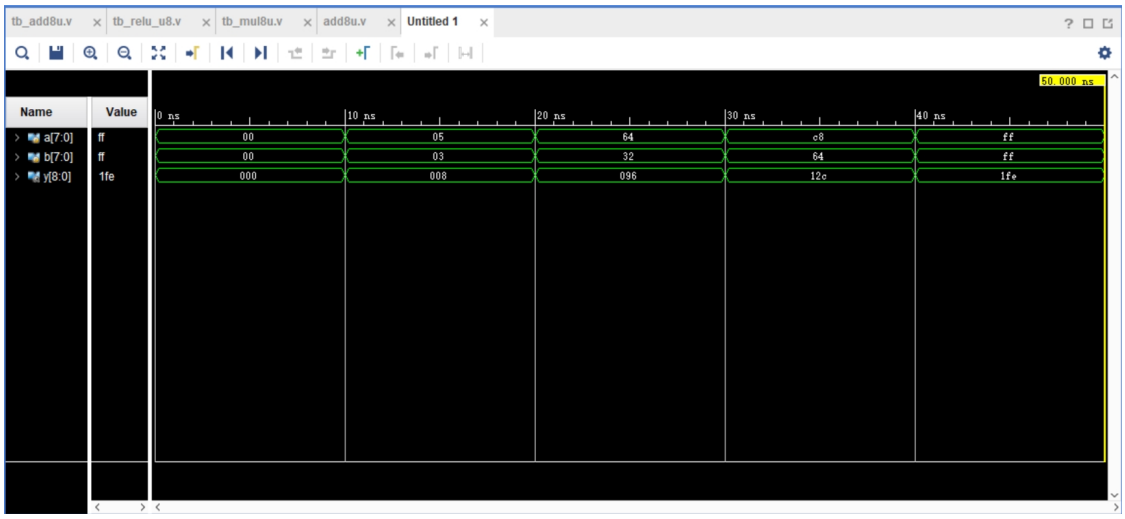


图 6.2: 8 位无符号加法器仿真波形

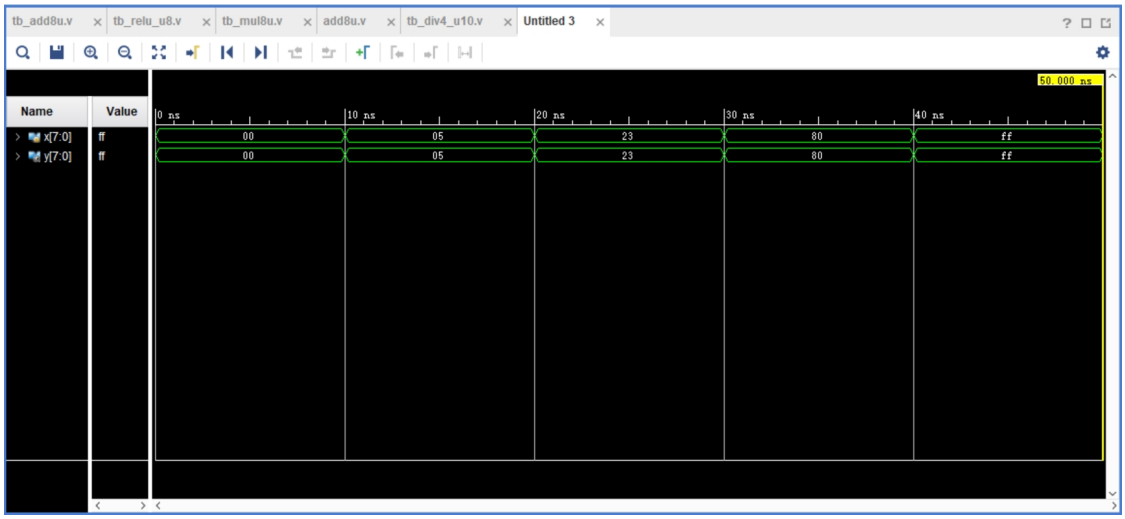


图 6.3: ReLU 激活模块仿真波形

本实验中 ReLU 模块输入为 8 位无符号数，因此输入数据不存在负数。仿真结果中，输入 `x` 与输出 `y` 始终保持一致，例如：

$$x = 05,\ y = 05$$

$$x = ff,\ y = ff$$

说明该模块能够实现无符号输入条件下的 ReLU 直通功能。

6.4 除以 4 模块仿真结果分析

图6.4为除以 4 模块的仿真波形。

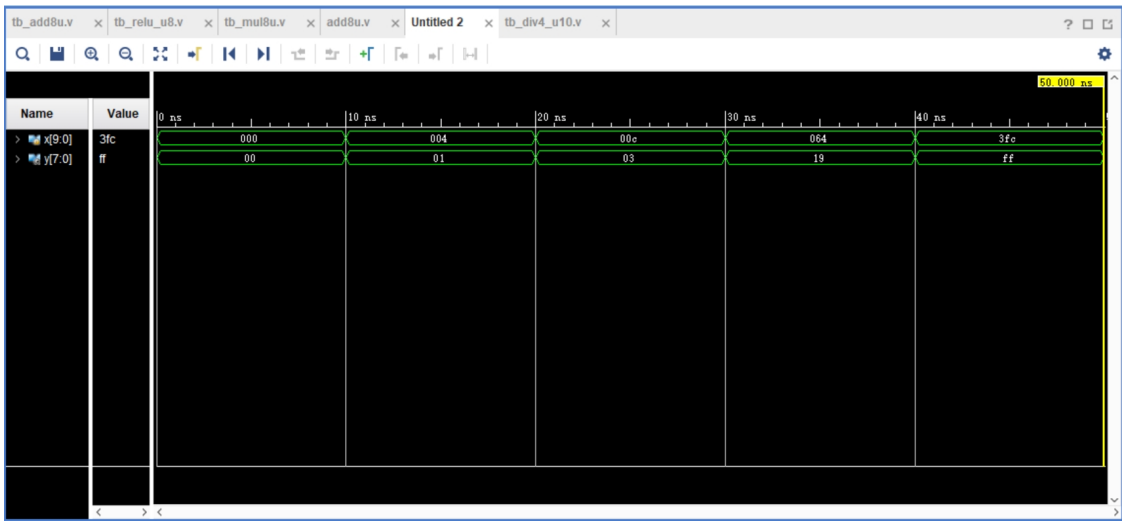


图 6.4: 除以 4 模块仿真波形

由仿真结果可知，当输入为 `x=004` 时，输出为 `01`，对应十进制结果为：

$$4/4 = 1$$

当输入为 `x=064` 时，输出为 `19`，其中 `064` 对应十进制 100，`19` 对应十进制 25，符合：

$$100/4 = 25$$

当输入为 `x=3fc` 时，输出为 `ff`，其中 `3fc` 对应十进制 1020，`ff` 对应十进制 255，符合：

$$1020/4 = 255$$

因此，除以 4 模块功能正确。

7 实验总结

本周完成了 8 位定点 CNN 运算器基础模块的设计与功能验证。实验分别实现了 8 位无符号乘法器、8 位无符号加法器、ReLU 激活模块和除以 4 模块。其中，乘法器用于卷积层中的像素与权重相

乘，加法器用于卷积累加和池化求和，ReLU 模块用于激活处理，除以 4 模块用于实现 2×2 平均池化。

通过 Vivado 行为仿真可以看出，各模块输出结果均与理论计算结果一致。乘法器能够正确输出最大结果 65025，加法器能够正确输出最大结果 510，ReLU 模块能够在无符号输入条件下实现输入直通，除以 4 模块能够通过右移 2 位正确完成平均计算。

综上所述，第一周基础运算模块设计完成，仿真结果正确，为后续实现完整的卷积-激活-池化链式运算器奠定了基础。

附录 A Verilog 源代码

A.1 8 位无符号乘法器代码

```
module mul8u (
    input  [7:0] a,
    input  [7:0] b,
    output [15:0] y
);

assign y = a * b;

endmodule
```

A.2 8 位无符号加法器代码

```
module add8u (
    input  [7:0] a,
    input  [7:0] b,
    output [8:0] y
);

assign y = a + b;

endmodule
```

A.3 ReLU 激活模块代码

```
module relu_u8 (
    input  [7:0] x,
    output [7:0] y
);

assign y = x;

endmodule
```

A.4 除以 4 模块代码

```
module div4_u10 (
    input  [9:0] x,
    output [7:0] y
);

assign y = x >> 2;

endmodule
```

A.5 乘法器测试文件代码

```
`timescale 1ns/1ps

module tb_mul8u;

reg  [7:0] a;
reg  [7:0] b;
wire [15:0] y;

mul8u uut (
    .a(a),
    .b(b),
    .y(y)
);

initial begin
    a = 0;   b = 0;
    #10;

    a = 8'd5;   b = 8'd3;     // 15
    #10;

    a = 8'd10;  b = 8'd10;    // 100
    #10;

    a = 8'd20;  b = 8'd15;    // 300
    #10;

    a = 8'd255; b = 8'd255;   // 65025
    #10;

    $finish;
end

endmodule
```

A.6 加法器测试文件代码

```
`timescale 1ns/1ps

module tb_add8u;

reg  [7:0] a;
reg  [7:0] b;
wire [8:0] y;

add8u uut (
    .a(a),
    .b(b),
    .y(y)
);

initial begin
    a = 0;   b = 0;
    #10;

    a = 8'd5;   b = 8'd3;     // 8
    #10;

    a = 8'd100; b = 8'd50;    // 150
    #10;

    a = 8'd200; b = 8'd100;   // 300
    #10;

    a = 8'd255; b = 8'd255;   // 510
    #10;

    $finish;
end

endmodule
```

A.7 ReLU 测试文件代码

```
`timescale 1ns/1ps

module tb_relu_u8;

reg  [7:0] x;
wire [7:0] y;

relu_u8 uut (
```

```
    .x(x),
    .y(y)
);

initial begin
    x = 8'd0;
    #10;

    x = 8'd5;
    #10;

    x = 8'd35;
    #10;

    x = 8'd128;
    #10;

    x = 8'd255;
    #10;

    $finish;
end

endmodule
```

A.8 除以 4 测试文件代码

```
`timescale 1ns/1ps

module tb_div4_u10;

reg  [9:0] x;
wire [7:0] y;

div4_u10 uut (
    .x(x),
    .y(y)
);

initial begin
    x = 10'd0;      // 0
    #10;

    x = 10'd4;      // 1
    #10;

    x = 10'd12;     // 3
    #10;
```

```
22
23      x = 10'd100;     // 25
24      #10;
25
26      x = 10'd1020;    // 255
27      #10;
28
29      $finish;
30  end
31
32  endmodule
```